

Computer Architectures notes

Luca Seggiani
2026

1 Lesson 22-09-26

1.1 Introduction

The aim of this course is to provide basic knowledge about the architecture of modern *microprocessor*-based systems. Special emphasis will be put on the *microprocessor core* itself.

The course is structured in two main blocks:

1. Modern processor architecture. This will cover computer design, the *RISC* processor architecture, superscalar processors and disrupting architectures;
2. *ARM*-based systems, that is *ARM* assembly language and *ARM Cortex-M3* based systems. Both development boards and emulation systems will be used to aid in the study of *ARM*-based systems.

1.2 CISC vs. RISC

We first go over what an **ISA** (*Instruction Set Architecture*) is in the context of processor design. Then we distinguish between **CISC** (*Complex Instruction Set Computer*) and **RISC** (*Reduced Instruction Set Computer*) architectures.

1.2.1 High level vs. Low level languages

Many programming languages exist. They can be classified into:

- **High-level** programming languages, which abstract over the machine to offer more sophisticated capabilities. This gives developers the freedom to express the same algorithm in many ways, and usually to express the same algorithm with significantly less lines of code than those used in low-level languages;
- **Low-level** programming languages, which stay closer to an **ISA** (*Instruction Set Architecture*). An ISA is a specification which defines:
 - Supported instructions;
 - Data types;
 - Registers;
 - Addressing modes.

In low-level languages, operations are smaller than those provided in high-level languages, so even simple algorithms usually take more instructions to express.

We can express this difference by taking the simple example of a loop:

1. In high-level languages such as *Python* and *C*, loops are expressed via a simple and predefined syntax which allows to compactly express iteration value initialization, update and the exit condition;
2. In a low-level language, all of these operations are distinct: the programmer has to explicitly initialize an iteration value, perform a comparison to check for the exit condition, and manually update the iteration value.

Another idea to note is that high-level languages are usually designed to be *portable*, that is *interpreted* or *compiled* for a variety of systems. On the other hand, low-level languages are usually designed for a single machine (a single type of processor), and porting to other platforms requires manual conversion work.

Programs written in high-level languages are usually compiled on the machines they are meant to run on: this means they are converted to low-level machine code which can be directly executed. For environments where running a full compiler is not feasible (such as a development board), a technique known as *cross-compilation* can be used: the program is compiled on a second machine, into machine code for the first machine. All of this of course doesn't apply to low-level work, where we are expected to write the same routines from scratch, and likely differently, for each target architecture.

1.2.2 Data types and addressing

An ISA also defines the **data types** the CPU can address. For instance, the RISC architecture might allow a programmer to address bytes, half words (double bytes) and words (double half words). With this being said, we note that usually low-level languages interpret all data as *binary* data (complex types such as *integer*, *float*, ... are usually added in as high-level constructs).

There are two main methods of performing addressing:

- **Register-Memory** architectures: operands of instructions can be immediates (contained within the instruction encoding), or contained in memory *or* in registers. If *all* operands can be in registers or in memory (or in combinations), the architecture is called *register-plus-memory*. Clearly, as memory accesses are also operations that have to be carried out by the processor, these kinds of ISAs are more complex.
- **Register-Register** architectures: these are also called **load-store** architectures. Instructions of register-register architectures are explicitly divided into:
 - Memory access (*load* and *store* between memory and registers);
 - **ALU** (*Arithmetic and Logic Unit*) operations (only between registers). The ALU itself is one of the most important components of a processor.

It follows that operands of ALU instructions are immediates or contained in registers.

In CISC (usually *register-to-memory*) instruction sets, the order of instructions can affect the number of memory accesses. The more the number of memory accesses increases, the more performance degrades. Load-store architectures, by their nature, do not suffer from this problem

1.2.3 CISC

CISC (*Complex Instruction Set Computer*) was the name given to the first computer designs. The main purpose is to complete a task in as few assembly lines as possible. This makes sense in an environment where programmers themselves are tasked with writing assembly code, as we said earlier that low-level languages tend to express the same concepts with more lines of code (we refer to this as a *semantic gap* between low and high-level languages).

Common examples of CISC CPUs are:

- x86 (1978) and x64 (2003) family, by Intel and AMD;
- Motorola 68000 series (1979);
- IBM System/360 family (1964);
- PDP-11 (1970) and VAX families (1977), by DEC.

Some characteristics of CISC are:

- Instruction encoding has *variable length*: for an immediate instruction we need to store the immediate, whereas for register to register or register to memory instructions we need to store register and memory addresses. As an example, x64 family instructions are between 1 and 6 bytes;
- The processor architecture has to be more *complex*, as a single instruction can express complex operations;
- The number of *clock cycles* required for execution of a single instruction is wildly variable: if an instruction requires a memory access, we need to wait for the memory to respond, which may happen on timescales far larger than those of the CPU;
- The resulting *code size* is far smaller than that of other architectures.

1.2.4 RISC

RISC (*Reduced Instruction Set Computer*) is the name given to a family of processors initially developed during the 1980s. The main purpose is to make the processor as simple as possible: this requires more work on the programmers' side, but allows the CPU to move much faster and execute many more instructions in the same amount of time as a CISC CPU. The limitation of more complex code, then, can be removed by making use of highly optimized *compiler* which take the toll of manual assembly programming away from programmers.

Common examples of RISC CPUs are:

- ARM family (1985), designed by ARM Holdings, which will also be the focus of the second part of this course;
- MIPS family (1984), by MIPS Technologies Inc.;
- RISC-V (2014), an example of a *register-to-register* ISA;
- SPARC family (1985), by Sun Microsystems;
- PowerPC (1991), by Apple, IBM, and Motorola.

Some characteristics of RISC are:

- The instructions themselves are *simpler*;
- The length of the instructions can be *fixed* (for example, in RISC-V they have a fixed size of 32 bits). A fixed instruction size sometimes requires some bits of the instruction encoding to remain unused: this is still preferred over variable length encoding as it allows the component of the CPU that performs the *fetch-decode* cycle to be far simpler;

- The processor architecture is *simpler*;
- The number of clock cycles required to execute an instruction is normally *lower* than in CISC, and there is less overall variance. This makes it easier to design and validate both RISC code and processors, as the amount of time taken by each instruction is easy to estimate (optimistically, 1 cycle per instruction).

Common factors that disallow having exactly 1 cycle per instruction involve:

- *Latencies* introduced by CPU optimizations, such as pipeline flushes and branch misses;
- *Memory accesses*: load and store instructions always have to at least wait for the cache, and if a miss happens, for the memory, which we said is usually slower than the CPU;
- Some *complex instructions* can be introduced back into the architecture: these give the programmer more power, but also require more clock cycles to complete.
- The resulting *code size* is larger than that of other architectures.

All of these characteristics contributed to making RISC architectures more popular than CISC architectures nowadays. In fact, some modern CISC processors actually convert the CISC instructions to an internal RISC representation, which then gets actually executed on the processor.

Also, the tradeoff given by longer code length is balanced by the fact that we are allowed to have *easier* CPUs, that is CPUs that are easier to *design* and *validate*. Some extensions have been introduced (such as the C "Compressed" extension for RISC-V) to make RISC-V instructions smaller and reduce the code memory footprint.

1.2.5 Comparison of addressing modes

As an example, let's compare the available addressing modes in x86 (a CISC architecture) and RISC-V (a RISC architecture). We then go over a specific detail we will encounter in ARM architectures.

- In x86, a number of addressing modes are available:
 - Register mode: `MOV AX, BX`
 - Immediate mode: `MOV AX, 2000`
 - Displacement or direct mode: `MOV AX, [DISP]`
 - Register indirect mode: `MOV AX, [DI]`
 - Based indexed mode: `MOV AX, [BX + DI]`
 - Indexed mode: `MOV AX, [SI + 2000]`
 - Based mode: `MOV AX, [BP + 0100]`
 - Based indexed displacement mode: `MOV AX, [SI + BP + 2000]`

Following extension then introduced **SIB** (*Scale-Index-Base*) addressing for arrays, *Mod R/M* bits.

The main idea is that an *Expected Address* (*EA*) is generally defined as:

$$EA = \text{Base} + \text{Index} \times \text{Scale} + \text{Displacement}$$

where:

- **Base** is a general-purpose register, containing a base memory address;
- **Index** is a general-purpose register containing a memory offset;
- **Scale** is a special constant, which can be 1, 2, 4, or 8;
- **Displacement** is an arbitrary constant, offered as an immediate in the instruction.

Any of these components can be omitted, resulting in the complex array of addressing modes shown above (the indexed/based move differ mainly in semantics because of historical reasons).

In the ISA, these addressing modes are specified by the instruction encoding, by the ModR/M byte, with an optional SIB byte. For more information, refer to the notes available at <https://raw.githubusercontent.com/seggiani-luca/appunti-rl/34228f66db395637bd1824d04f3130b977cc0ce4/master/master.pdf> (in italian).

- In **RISC-V**, the same operations is significantly simpler. We have available:
 - ALU operations:
 - * Register addressing mode: `ADD t0, t1, t2`
 - * Immediate addressing mode: `ADDI t0, t1, 10`
 with the first register taken as the destination, in common RISC fashion.
 - Load / store operations, in base + displacement addressing mode: `LW t2, 4(t1)`, where the constant before the open parenthesis is an immediate offset, and the register contains a memory address. Of course, since for instant RV32x is a 32 bit architecture, 32 bit instructions cannot fully contain immediate addresses: this is taken into account by special facilities in the processor which allow full address loading via multiple instructions.
- The same is available in ARM, with the distinction of having *pre-indexed* and *post-indexed* memory operations, that is control over the time when the base register is updated.

Being more specific, ARM allows load and store instructions to optionally *alter* the address contained within a base register. 3 forms are available:

- `ldr r0, [r1, #4]` is equivalent to immediate offset addressing in RISC-V, with #4 being the immediate operand;
- `ldr r0, [r1, #4]!`, with the additional `!`, means performing a *write-back* or **pre-indexing** operation. The calculated address, obtained by adding the offset, is *written back* into the base register before the load operation. The result we get is that of a load from the base address plus the offset, with the base address reflecting the change after execution. Compared to RISC-V, this saves an instruction;
- `ldr r0, [r1], #4` means performing a **post-indexing** operation. This means increasing the value of the base register by offset, after performing the load. We can think of this as being analogous to the `++` syntax available in C. This also saves an instruction compared to RISC-V.

As we can see, the main point of this complication is to save a few instructions when accessing arrays, or generally where striding memory access patterns are needed in ARM architectures.